Main memory

Relocation, segmentation, paging, and page-table structures connect address translation to protection, fragmentation, TLB behavior, and effective access time.

Prerequisites. Binary addresses, Registers and the memory interconnect, Process isolation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Derive base-and-limit, segment, and page translations
- Distinguish internal and external fragmentation
- Calculate TLB costs and explain page-table levels

Central question. How can a logical address space map safely onto finite, physically scattered memory?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.1. Background

- The MMU translates CPU-generated logical addresses to physical addresses under protected configuration. Ordinary processes do not gain unrestricted physical access by choosing an integer address.
- In a base-and-limit model, a legal offset d satisfies 0<=d<limit and translates to base+d. Hardware checks accesses; untrusted software's prior check cannot enforce the boundary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.1. Model, ordering, and boundary cases

Model assumptions. The CPU uses logical addresses and hardware enforces the selected relocation or paging model.

Invariant and ordering. A segment offset is less than its limit. A k-bit page offset has numeric range 0 through 2^k-1; comparing an address value with the bit count k would be a unit error.

Counterexample or boundary. Trusting a user program to validate its own pointer lets hostile code bypass the check.

Worked check. With base=0x8000 and limit=0x2000, offset 0x1FFF is legal and resolves to 0x9FFF. Offset 0x2000 is outside the half-open range.

Reasoning prompt. How does placing protection in trusted hardware rather than untrusted software affect the trust model?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.2. Swapping

- Classical swapping moves an entire process image to backing storage to free RAM. Restoration needs the saved image, address-space metadata, and an appropriate placement or relocation mechanism.
- Demand paging selectively loads pages rather than requiring the whole image. Its granularity and cost differ from whole-process swapping.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.2. Model, ordering, and boundary cases

Model assumptions. Backing storage has sufficient capacity and the manager retains the required process state.

Invariant and ordering. Only one image is authoritative during a move. A frame used by outstanding DMA cannot be reassigned while the device may still access it.

Counterexample or boundary. A DMA buffer swapped out and reused for another process can receive a later device write into unrelated data.

Worked check. An 80 KiB image fits a 100 KiB free backing area. Before resuming the restored process, install the correct mappings and invalidate translations as required by the architecture.

Reasoning prompt. Which control points and costs differ between a whole-image swap and a demand-page transfer?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.3. Contiguous allocation

- Each allocation occupies a contiguous physical region in this model. First-fit selects the first sufficient hole in traversal order; best-fit selects the smallest sufficient hole; worst-fit selects the largest.
- Allocations and frees create external fragmentation. Compaction moves allocated regions and requires relocation support and coordination with address-dependent activity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.3. Model, ordering, and boundary cases

Model assumptions. The free list contains only usable holes. Updates are synchronized, and growth or maximum-size rules are explicit.

Invariant and ordering. Allocated ranges do not overlap and allocated plus free ranges account for the managed region. Movement preserves references and protected access.

Counterexample or boundary. Four separate 5 KiB holes total 20 KiB yet cannot satisfy one contiguous 10 KiB request.

Worked check. In free-list order 100,500,200 KiB, a 180 KiB request uses the 500 KiB hole under first-fit, leaving 320. Best-fit uses 200, leaving 20. Worst-fit also uses 500. Selecting 200 for first-fit would ignore the given traversal order.

Reasoning prompt. Which allocation pattern leaves many unusably small holes under best-fit?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.4. Segmentation

- A logical address contains segment number and offset. A segment-table entry supplies base, limit, and applicable rights. A legal access translates to base+offset after bounds and permission checks.
- Segments represent logical units with variable lengths and can suffer external fragmentation. Fixed-size paging avoids that placement problem at page granularity but can waste space inside a final page. The mechanisms can be combined.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.4. Model, ordering, and boundary cases

Model assumptions. Segment bounds and lifetimes are tracked, and hardware faults on unauthorized or out-of-range accesses.

Invariant and ordering. Offset is below limit and the resulting access respects the authorized segment mapping. Explicitly shared segments are a separate permitted case.

Counterexample or boundary. Checking base without checking the requested offset and size permits an access into an adjacent segment.

Worked check. Segment 2 has base=0x40000 and limit=0x3000. Offset 0x2FFF maps to 0x42FFF; 0x3000 faults for a single-byte access.

Reasoning prompt. Why can a logical segmentation model still use paging to simplify physical placement?

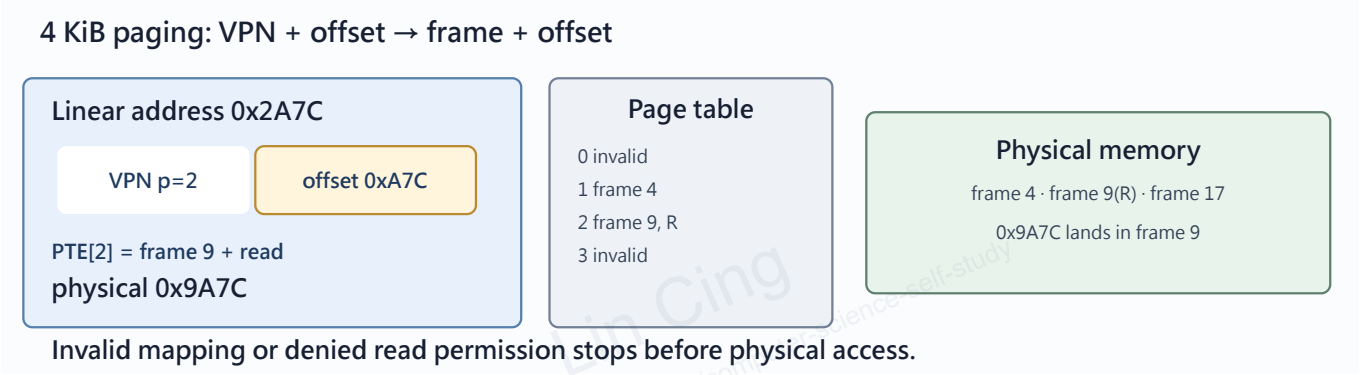
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5. Paging

- Split a virtual address into page p and offset d. If an authorized resident entry maps p to frame f, physical address=f*page_size+d. The offset is unchanged.
- Smaller pages reduce tail waste but increase the number of entries for a fixed range. Larger pages increase TLB coverage per entry and may reduce table work, while increasing allocation and transfer granularity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5. Paging (continued)



Original vector diagram for this course.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5.1. Internal and external fragmentation

- Internal fragmentation is unused space inside an allocated unit that exceeds the request. Fixed pages and filesystem blocks can create it.
- External fragmentation leaves enough total free space in separate holes but no contiguous region large enough for one request.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5.2. TLBs and effective access time

- A TLB caches translations. In a serialized lookup model, a hit costs t+m. A miss requiring k page-table memory reads costs t+k*m+m, including the eventual data access.
- With hit fraction h, EMAT=h(t+m)+(1-h)(t+k*m+m). A parallel TLB/cache lookup changes the timing model; page faults and additional cache levels require separate terms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5.3. Protection and sharing

- Entries may encode read, write, execute, privilege, and presence rules. Different address spaces can share a physical frame through explicitly authorized mappings.
- Read-only code sharing saves frames. An accidentally writable or executable mapping can break isolation. Presence alone does not grant every operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.5. Model, ordering, and boundary cases

Model assumptions. Page size is 2^k and descriptors contain architecture-specific mapping, presence, and permission state. Translation caches follow an invalidation protocol.

Invariant and ordering. Successful access has a valid translation and sufficient rights. Revocation or frame reuse cannot leave an old translation usable in an affected address space.

Counterexample or boundary. Unmapping a page without the required TLB invalidation may let a CPU reach a frame already reused by another process.

Worked check. With 4 KiB pages, frame 9 and offset 0xA7C give 0x9000+0xA7C=0x9A7C.

Reasoning prompt. Which mapping changes also require TLB or cache maintenance?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.6. Page-table structures

- Large virtual ranges make dense linear tables expensive. Hierarchical tables omit unused lower levels; hashed tables and inverted tables organize lookup differently.
- More levels save sparse-table storage but can add work on a TLB miss. A 64-bit instruction set need not implement all 64 bits of virtual address space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.6. Model, ordering, and boundary cases

Model assumptions. Each hierarchical entry points to the next valid level or an allowed leaf. Address fields follow the chosen mode.

Invariant and ordering. A page walk must resolve every required level before reaching a frame. A missing intermediate mapping cannot be treated as a usable translation.

Counterexample or boundary. Building a fully dense table for every possible 64-bit byte address ignores both sparsity and implemented address-width limits.

Worked check. Linear 0x12345678 with 4 KiB pages splits PDI 0x048, PTI 0x345, offset 0x678. If outer[0x048] points to T and T[0x345] to frame 0xABC, PA=0xABC678. An absent inner entry faults; VA cannot be used as PA.

Reasoning prompt. Which miss costs accompany the storage saved by a hierarchical table?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

IA-32 linear-address walk

Fix protected mode, ordinary 4 KiB pages, and linear address 0x12345678 after segmentation. The selector first locates an LDT/GDT descriptor; segment base/limit and rights produce the linear address. Paging translates that result, so the given number is not the pre-segmentation logical address.

A 4 KiB directory/table has 1024 four-byte entries, giving 10/10/12 bits. For linear L,

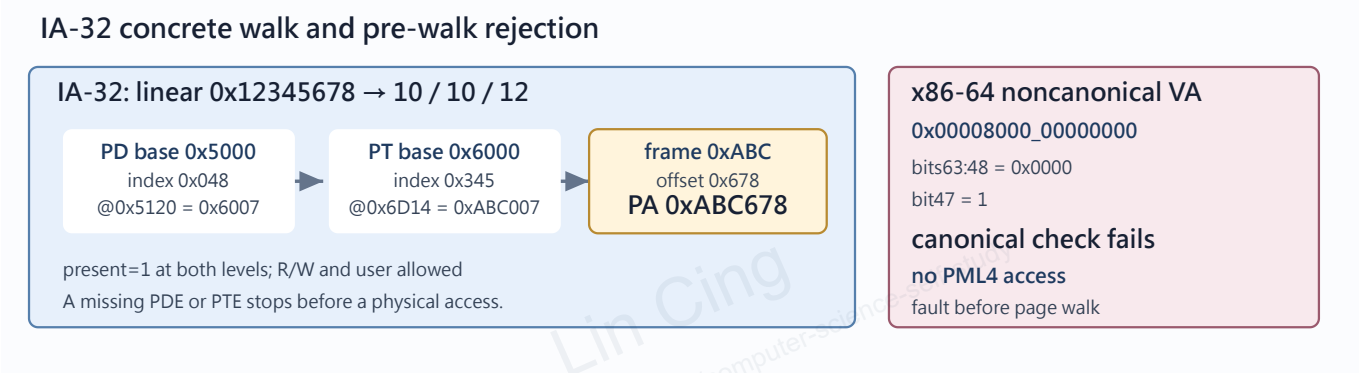
$$PDI = (L \gg 22)\ \&\ 0x3FF, \quad PTI = (L \gg 12)\ \&\ 0x3FF, \quad d = L\ \&\ 0xFFF.$$

CR3 base 0x5000 and next PT base 0x6000 yield:

Level	Index	Entry address	Entry value	Meaning
Directory	0x048	0x5000+4×0x048=0x5120	0x00006007	Present, next base 0x6000, RW/user
Table	0x345	0x6000+4×0x345=0x6D14	0x00ABC007	Present, frame 0xABC, RW/user

Offset 0x678 gives PA0xABC000+0x678=0xABC678. This path uses directory, table, and target page, each 4 KiB. Table-allocation costs are separate from an individual walk.

IA-32 linear-address walk (continued)



An absent PDE stops before reading the PT; an absent PTE cannot supply a usable frame. A4 MiB PDE with Page Size set terminates at the directory and uses 22 offset bits instead. That is a different encoding from this fixed trace.

Canonical x86-64 addresses and four levels

Fix 48-bit canonical virtual addresses, four levels,4 KiB pages,8-byte entries. Index fields are[47:39], [38:30],[29:21],[20:12], with offset[11:0]. Each table has 512 entries. First require 0≤v<2^64, then require every bit 63:48 to repeat bit 47.

$$canonical(v) \iff (v \gg 48) = \begin{cases} 65535, & ((v \gg 47)\ \&\ 1) = 1, \\ 0, & ((v \gg 47)\ \&\ 1) = 0. \end{cases}$$

Comparing only bit 63 to bit 47 is insufficient. v=0x0001_0000_0000_0000 has both 0 but upper 16 bits=1, so is noncanonical. Flipping any one sign-extension bit of a legal address invalidates it. Ranges 0…0x00007FFFFFFFFFFF and 0xFFFF800000000000…0xFFFFFFFFFFFFFFFF are legal; the interval between is illegal.

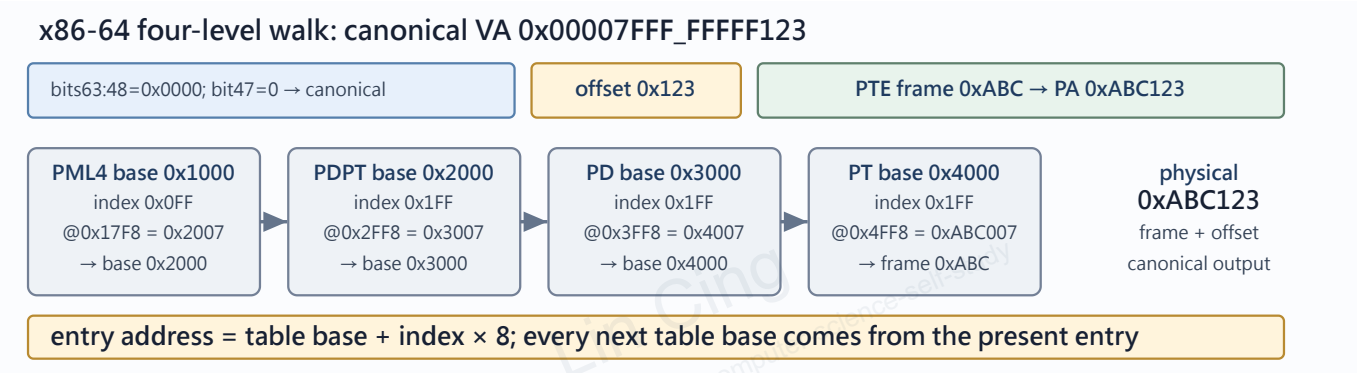
For VA 0x00007FFF_FFFFF123, bit 47=0 and upper 16 bits=0. CR3=0x1000; subsequent bases are 0x2000,0x3000,0x4000.

Canonical x86-64 addresses and four levels (continued)

Level	Index	Offset	Entry address	Entry value	Next base/frame
PML4	0x0FF	0x7F8	0x17F8	0x2007	PDPT 0x2000
PDPT	0x1FF	0xFF8	0x2FF8	0x3007	PD0x3000
PD	0x1FF	0xFF8	0x3FF8	0x4007	PT0x4000
PT	0x1FF	0xFF8	0x4FF8	0xABC007	Frame 0xABC

Every address is base+index×8; every next base comes from the present entry. Offset 0x123 is retained, so PA=0xABC123.

Canonical x86-64 addresses and four levels (continued)



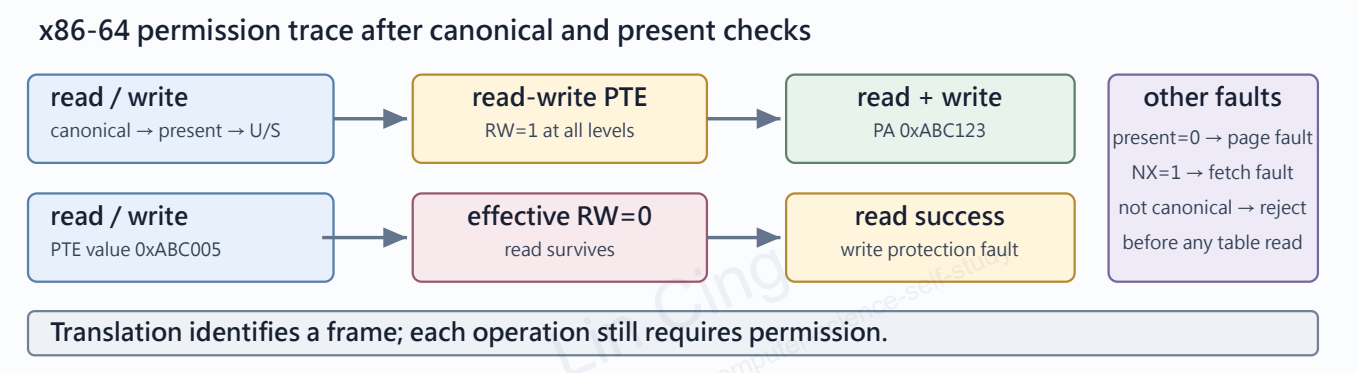
2 MiB and 1 GiB encodings end higher in the walk and require other formats/alignment and larger offsets. The 9/9/9/9/12 split applies only to this 4 KiB configuration.

Mapping and operation permission

Canonical check precedes all table memory accesses. Present establishes usable entries, U/S restricts user access at every level, effective RW restricts writes, and NX restricts instruction fetch.

Case	Canonical/mapping	Permissions	Read	Write	Fetch
Normal trace	Canonical, four present levels	All user/RW, NX0	0xABC123	0xABC123	Allowed
PTE 0xABC005	Same present path	User, RW0, NX0	0xABC123	Protection fault	Allowed
PML4[0x0FF] absent	Canonical, no mapping	No effective mapping	Page fault	Page fault	Instruction page fault
PTE NX1	Canonical, present	User/RW, NX1	Allowed	Allowed	Fetch fault
VA 0x00008000_00000000	Noncanonical	No walk	Reject	Reject	Reject

Mapping and operation permission (continued)



Mapping and operation permission (continued)

The last address has bit 47=1 but upper 16 bits=0. The canonical address 0x00007FFF_FFFFF000 still page-faults if any required level is absent. The illegal interval ends at 0xFFFF7FFFFFFFFFFF, immediately below the upper canonical range. A root, frame, and valid canonical input do not alone authorize every operation. PAE, five levels, large pages, or another physical width require the matching masks and entries. Updated entries need required TLB invalidation; a stale cached permission can outlive a changed PTE.

ARMv7-A short descriptors

Fix AArch32, short descriptors, TTBCR. N=0, SCTLR. AFE=0, SCTLR. TRE=0, PL1 table construction, and 4 KiB small pages. This is not AArch64 long-descriptor translation. N=0 selects TTBR0 for all input VAs. The complete L1 has 4096 four-byte entries,16 KiB, aligned 16 KiB. Sparse use does not permit allocating only the populated entries; other entries need valid architectural encodings, including invalid ones.

$$L1addr = TTBR0 + ((VA \gg 20)\ \&\ 0xFFF) \times 4.$$

A coarse L1 descriptor points to a256-entry L2,1 KiB, aligned 1 KiB; bits 31:10 supply its base.

$$L2addr = L2base + ((VA \gg 12)\ \&\ 0xFF) \times 4, \qquad PA = frame + (VA\ \&\ 0xFFF).$$

ARMv7-A short descriptors (continued)

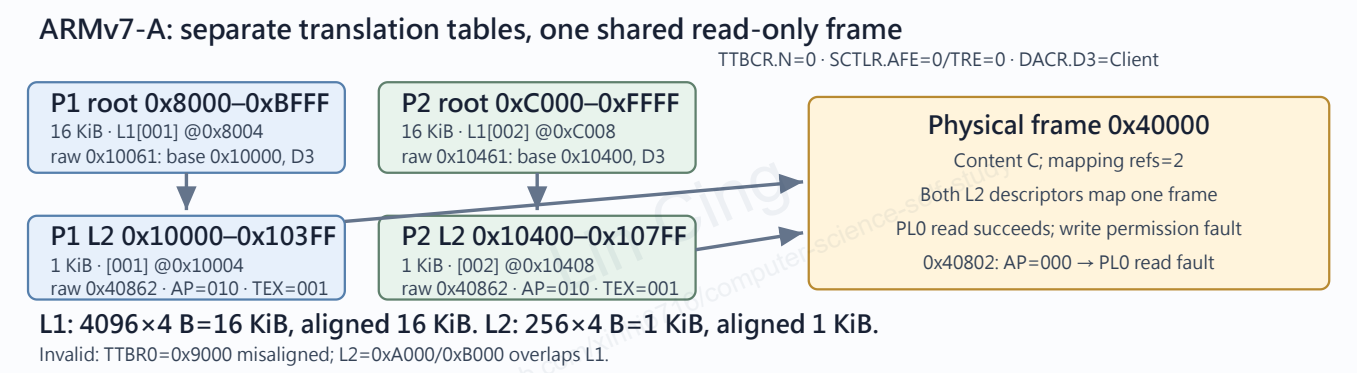
Storage	Base	Inclusive range	Size/alignment
P1 root	0x8000	0x8000–0xBFFF	16 KiB, base mod 0x4000=0
P2 root	0xC000	0xC000–0xFFFF	16 KiB, base mod 0x4000=0
P1 L2	0x10000	0x10000–0x103FF	1 KiB, base mod 0x400=0
P2 L2	0x10400	0x10400–0x107FF	1 KiB, base mod 0x400=0

All four ranges are distinct. Only the final data frame is shared.

Process/VA	L1 index/address	L2 base	L2 index/address	Frame	PA
P1 0x00101000	0x001 /0x8004	0x10000	0x001 /0x10004	0x40000	0x40000
P2 0x00202000	0x002 /0xC008	0x10400	0x002 /0x10408	0x40000	0x40000
P1 0x00101A5C	0x001 /0x8004	0x10000	0x001 /0x10004	0x40000	0x40A5C

ARMv7-A short descriptors (continued)

Offset[11:0]=0xA5C does not enter either table index. Keeping only 8bits would incorrectly give 0x4005C.



Raw ARM words, domains, AFE and AP

Both L1 descriptors use domain 3; DACR. D3=01 means Client. Domain is in L1 and inherited by L2. With AFE=0, small-page AP[2] is bit 9 and AP[1:0] bits 5:4, not section bits 11:10.

Descriptor	Raw	Decode
P1 L1	0x10061	Type 01 coarse, base 0x10000, domain 3
P2 L1	0x10461	Type 01 coarse, base 0x10400, domain 3
Both L2	0x40862	Frame 0x40000, type 10, XN0, nG1, AP010, TEX001, C=B=0

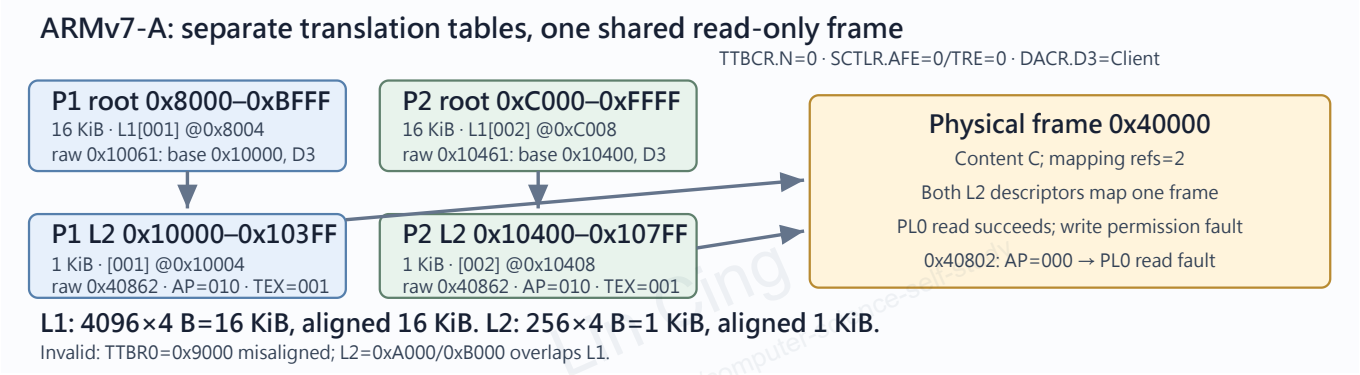
Decode field=(raw>>shift)&mask. AP2=(raw>>9)&1, AP1=(raw>>5)&1, AP0=(raw>>4)&1, TEX=(raw>>6)&7, C=(raw>>3)&1, B=(raw>>2)&1, XN=raw&1. AP010 gives PL1 read/write and PL0 read-only. With TRE0, TEX001/C0/B0 means Normal non-cacheable. Fetch additionally assumes WXN=UWXN=0, readable mapping, Client domain, and no other restriction. XN0 alone cannot prove executable Strongly-ordered memory.

Raw ARM words, domains, AFE and AP (continued)

0x40802 instead decodes AP000 and denies PL0 read/write even though the frame is valid. 0x40022 has AP010 but TEX/C/B=0/0/0, Strongly-ordered; it only illustrates data AP, not guaranteed fetch.

Mode	PL0 read	PL0 write	Reason
AFE0, Client, AP010	Success	Permission fault	Read-only PL0
AFE0, No access domain	Domain fault	Domain fault	Domain rejects first
AFE0, Manager domain 11	Not rejected by AP	Not rejected by AP	Manager bypasses AP; other faults remain possible
AFE1, Client, same 0x40862	Access-flag fault	Access-flag fault	AP0 is now access flag 0; software-managed flag model
AFE1, Client,0x40872	Success	Success	Flag 1, AP[2:1]=01 grants PL0 RW

Raw ARM words, domains, AFE and AP (continued)



Changing DACR or AFE changes the interpretation, so the AFE0 read-only result cannot be reused unchanged.

ARM unmap, reference counts and stale translations

Initially the frame contains C, mapping refcount 2, both entries 0x40862, Client parents, and no cached translation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ARM unmap, reference counts and stale translations (continued)

State/operation	Translation/permission	Result	Content/references
S0 P1 read	0x00101000→0x40000, AP permits	Success	C, refs 2
S0 P1 write	Valid mapping, AP denies	Permission fault	C unchanged
S0 P2 read	0x00202000→0x40000, AP permits	Success	C, refs 2
S0 P2 write	Valid mapping, AP denies	Permission fault	C unchanged
S1 remove P1	L2 invalid, required TLB maintenance complete	P1 translation fault	P2 remains, refs 1
S2 P2 read	Independent tables still valid	Success	refs 1
S3 remove P2	L2 invalid, maintenance complete	P2 translation fault	refs 0; reclaim only after all stale translations are unusable

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ARM unmap, reference counts and stale translations (continued)

An absent mapping is not a valid-but-read-only mapping. Descriptor changes need architecture-specific VA/ASID TLB maintenance and barriers. nG1 associates translations with their ASIDs; ASID does not determine domain or AP. Clearing a PTE alone cannot authorize frame reuse while an old TLB can still access it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Table storage must be valid before sharing is analyzed

Invalid layout	Failure
P2 TTBR0=0x9000, N0	0x9000 mod 0x4000=0x1000, root misaligned
P1 root 0x8000, P2 root 0x9000	Ranges 0x8000–BFFF and 9000–CFFF overlap
P1 L2=0xA000	Inside P1 root
P2 L2=0xB000	Inside P1 root
Only 4 KiB allocated for L1	4096×4 entries require 16 KiB

Legal sharing has independent roots/L2 storage and one shared frame. Verify page identity, storage, domain/AP/XN, references, and TLB state together. A writable P1 mapping would modify the frame P2 reads; that is writable sharing, not read-only code. Preserving P2's old data while P1 changes requires COW or a separate frame, not merely changing permissions in one entry.

8.9. Summary

- Segmentation organizes logical objects; paging simplifies physical placement. Both require defined protection and sharing semantics.
- TLB lookup, page walks, and fault handling form one access path. Mapping changes must invalidate obsolete interpretations before unsafe reuse.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.9. Model, ordering, and boundary cases

Model assumptions. Hardware checks accesses and fault entry preserves the architecture's recovery state.

Invariant and ordering. Mapping, presence, rights, and cached translations agree at the points required for successful access and safe reclamation.

Counterexample or boundary. Changing a frame number while using the wrong offset width produces a different physical address even if the frame exists.

Worked check. An invalid range is rejected, a forbidden write is denied or handled under an explicit mechanism such as COW, and a TLB miss may simply trigger a successful page walk.

Reasoning prompt. Which faults can the OS service, and which accesses must it reject?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter conclusions

- A mapping and its permissions are distinct parts of access validation
- Classify fragmentation before comparing allocation schemes
- Effective access time is a model with explicit cache and fault assumptions

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exercises and sources

- the separate exercises PDF.
- the separate solutions PDF.
- Coverage and source limitations.

Reference: A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 9th edition, Wiley, 2013. These are original explanations and original exercises. Historical product cases refer to the historical versions in the edition. No publisher solution text is used.

- Intel SDM.
- ARMv7-A/R manual DDI0406C. d, B3.5 and B3/B4 descriptor, domain, and AFE definitions.
- Linux v3.2 ARM helpers. Fixed physical bases here are educational layouts rather than machine constants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study